SOLUTION DESIGN

Cross-Subject Motor Imagery EEG Classification Using Deep Learning

Methodology and implementation plan for a small hybrid CNN–Transformer that has to work for people it has never seen

TEAM	Ananya Gourishetty Haarika D Siddhanth Samal Sathvik P
GROUP	Group 5
GUIDE	Dr. Mohan Allam
COURSE	CSE4006 Deep Learning
SCHOOL	School of Computer Science and Engineering, VIT-AP University
DATASET	BCI Competition IV dataset 2a

Contents

Summary **4**
1. Introduction **4**
1.1 The problem 4
1.2 Why it matters 4
1.3 What we propose 4
2. Background and related work **5**
2.1 Motor imagery and the EEG signal 5
2.2 The four models we build on 5
2.3 How published models do on a new person 6
3. Research gap and objectives **8**
3.1 The gap 8
3.2 Objectives 8
4. Dataset **8**
4.1 Why BCI Competition IV 2a 8
4.2 What is in it 9
4.3 One trial, start to finish 9
4.4 The four classes 10
4.5 What the data mean for our design 10
5. Proposed solution **11**
5.1 Design principles 11
5.2 Approaches we considered 11
5.3 How the pieces fit 12
6. System architecture **12**
6.1 From raw EEG to a prediction 12
6.2 Software modules 13
6.3 What is learned and what is fixed 14
7. Preprocessing **14**
7.1 The eight steps 14
7.2 Euclidean alignment 15
7.3 Data augmentation 16
7.4 Keeping the test person out 16
7.5 Checks before any training 17
8. HCT-Net **17**
8.1 Constraints 17
8.2 Architecture 18
8.3 Parameter budget 19
8.4 Following one trial through the network 19
8.5 Why two encoder layers, not six 20
9. Training and evaluation **21**
9.1 Training set-up 21
9.2 Leave-one-subject-out 22
9.3 Why a validation person and not a random split 22
9.4 A within-subject check 22
9.5 Metrics 23
9.6 Deciding whether a difference is real 23
10. Experiments **24**
10.1 What varies and what stays fixed 24
10.2 The experiments 24
10.3 The component study 24
10.4 Result tables, fixed in advance 25
11. Implementation plan **26**
11.1 Tools 26
11.2 Order of work 26
11.3 Timeline 27
11.4 Milestones 27
12. Expected outcomes, risks and contribution **28**
12.1 What we expect 28
12.2 Risks and how we will respond 28
12.3 What is new, and what is not 28
13. Future work **29**
References **29**

Figures

Figure 1. A schematic of event-related desynchronisation. 5
Figure 2. CTNet's published accuracy on people seen in training against people held out, on two datasets. 6
Figure 3. Model size against accuracy on an unseen person, for the models re-run by the CTNet authors. 7
Figure 4. The timeline of one trial and the window we analyse. 10
Figure 5. The four imagined movements and where each is expected to appear on the scalp. 10
Figure 6. The system end to end. 12
Figure 7. What alignment does, sketched for three people. 15
Figure 8. The HCT-Net architecture. 18
Figure 9. The leave-one-subject-out fold structure. 22
Figure 10. The order and relative length of the implementation stages over the eighteen days. 27

Tables

Table 1. The four architectures from our literature survey and what HCT-Net borrows from each. 6
Table 2. Published leave-one-subject-out accuracy (%) on BCI IV-2a, from Zhao et al. [3]. 7
Table 3. Published within-subject accuracy on BCI IV-2a, with the protocol behind each number. 7
Table 4. Datasets we considered. 9
Table 5. Properties of BCI Competition IV dataset 2a [6]. 9
Table 6. Our design principles. 11
Table 7. Model families we considered. 11
Table 8. What each stage does. 13
Table 9. Software modules. 13
Table 10. Trainable and fixed parts of the system. 14
Table 11. The preprocessing pipeline. 14
Table 12. Checks on the data before any model is trained. 17
Table 13. Constraints fixed before the design. 17
Table 14. Layer-by-layer specification of HCT-Net. 18
Table 15. The parameter budget, worked out by hand. 19
Table 16. Parameter counts of HCT-Net and the published models. 19
Table 17. Training settings. 21
Table 18. Evaluation metrics. 23
Table 19. Variables and controls. 24
Table 20. The planned experiments. 24
Table 21. Component-study variants. 25
Table 22. Template for the main result: accuracy (%) on each unseen person. 25
Table 23. Tools and frameworks. 26
Table 24. Milestones and what counts as done. 27
Table 25. Expected outcomes. 28
Table 26. Risks and the responses we have planned. 28

Abbreviations

ALS	amyotrophic lateral sclerosis	**GELU**	Gaussian error linear unit
BCI	brain–computer interface	**GPU**	graphics processing unit
CNN	convolutional neural network	**GRU**	gated recurrent unit
CUDA	NVIDIA's GPU computing platform	**HCT-Net**	Hybrid Convolution–Transformer Network
EEG	electroencephalography	**LDA**	linear discriminant analysis
ELU	exponential linear unit	**LSTM**	long short-term memory
EOG	electrooculography	**MOABB**	Mother of All BCI Benchmarks
ERD	event-related desynchronisation	**SD**	standard deviation
FBCSP	filter-bank common spatial patterns	**YAML**	a plain-text configuration format

Summary

This is our solution design. Decoders of imagined movement work well on people they were trained on and much worse on anyone new, and that gap is what keeps brain–computer interfaces tied to long calibration sessions. We propose HCT-Net, a network of 20,996 parameters that joins EEGNet's convolutional front end to a two-layer Transformer attending within short windows, with Euclidean alignment of each person's data before training. We will test it against EEGNet, ATCNet, EEG Conformer, CTNet and FBCSP on BCI Competition IV dataset 2a, with every model under the same leave-one-subject-out protocol, three random seeds and paired statistics. A component study will show what each part contributes. The plan fits an eighteen-day implementation phase, and the data pipeline and baselines are verified before our own model is written.

Keywords: electroencephalography, motor imagery, brain–computer interface, cross-subject decoding, leave-one-subject-out evaluation, Transformer, Euclidean alignment

1. Introduction

1.1 The problem

When someone imagines moving their left hand, the electrical rhythm over the right side of their motor cortex changes, and electroencephalography (EEG), recorded through a cap of electrodes on the scalp, can pick that change up. A brain–computer interface (BCI) that reads these changes can turn imagined movements into commands without any physical movement. The catch is that EEG looks different from one person to the next. A decoder trained on one group of people usually does much worse on someone new, so in practice every new user first records a long labelled calibration session before the system works for them. This project is about removing that step. We want a decoder that classifies four imagined movements (left hand, right hand, both feet and tongue) for a person it has never seen.

1.2 Why it matters

The people who would gain most from a working BCI are the ones for whom other ways of giving input fail: people with advanced amyotrophic lateral sclerosis (ALS), a high spinal cord injury or a brainstem stroke. For them, a long calibration session at every visit is a real cost, and one that has to be repeated whenever the cap is moved is worse. A decoder that works on a new person from the start would make BCIs far easier to use outside a lab. It would also help in rehabilitation, where detecting the intention to move can trigger assisted movement, and in any setting where hands-free control is useful.

That is why we treat accuracy on unseen people as the main result of this project, not as a side note.

1.3 What we propose

We propose HCT-Net, a Hybrid Convolution–Transformer Network with 20,996 parameters. It uses the convolutional front end of EEGNet, a small Transformer encoder that attends within short overlapping windows, and Euclidean alignment of each person's data before training. We will test it, together with four published deep models and one classical method, under one leave-one-subject-out protocol on BCI Competition IV dataset 2a. The rest of this document explains the reasoning, the design, and how we will build and test it.

2. Background and related work

2.1 Motor imagery and the EEG signal

Imagining a movement suppresses the mu rhythm (about 8 to 13 Hz) and the beta rhythm (about 13 to 30 Hz) over the part of the motor cortex that controls that body part [13]. The effect is called event-related desynchronisation (ERD). For the hands it is lateralised: left-hand imagery weakens the rhythm over the right hemisphere, near electrode C4, and right-hand imagery over the left, near C3 (Figure 1). Foot imagery shows up near the midline, and tongue imagery lower down on both sides.

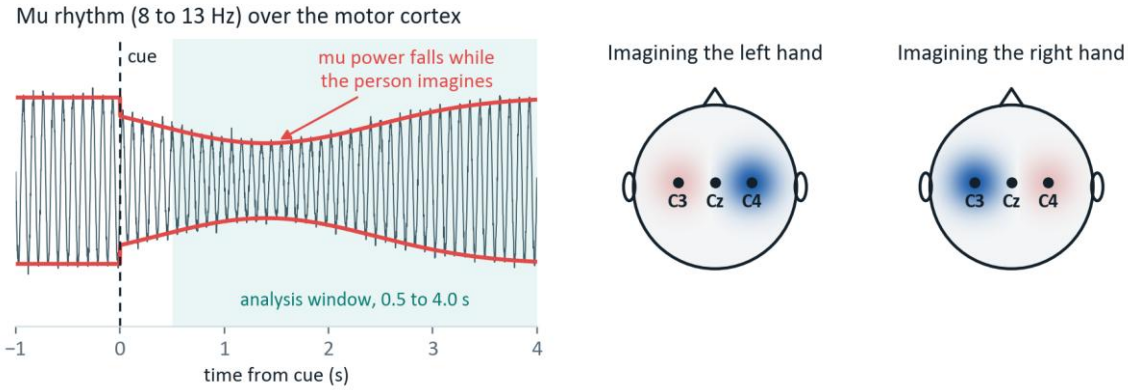


Figure 1. A schematic of event-related desynchronisation.

Left: the mu rhythm loses power after the cue while the person imagines the movement. Right: where that loss appears for left-hand and right-hand imagery (blue means weaker). Schematic, not recorded data.

Reading this signal is hard for three reasons. It is small, a few microvolts, and it sits under larger signals from eye movements, muscles and the mains supply. The data are short and wide: one four-second trial from 22 electrodes at 250 Hz holds more than twenty thousand numbers, yet a person can only produce a few hundred labelled trials before tiring. And the signal shifts between sessions and between people, which is the problem this project targets.

2.2 The four models we build on

Early motor imagery decoders used hand-designed features, most often common spatial patterns or its filter-bank version FBCSP [9], followed by a linear classifier. Deep learning replaced the hand-designed features with

networks that learn their own filters, first with compact convolutional networks [1], [8] and later with attention and Transformer layers [2]–[4], [10]. Our literature survey settled on four models that trace this path (Table 1).

Table 1. The four architectures from our literature survey and what HCT-Net borrows from each.
Parameter counts are from the braindecode 1.8.1 reference implementations for 22 channels, four classes and our 875-sample input.

Model	Parameters	Main idea	What we take
EEGNet (2018) [1]	3,188	A temporal convolution learns frequency filters, a depthwise convolution learns spatial filters over the electrodes, and a separable convolution summarises	The whole convolutional front end
ATCNet (2023) [2]	113,732	Attention and temporal convolution applied inside sliding windows of the feature sequence	Attention restricted to short windows
EEG Conformer (2023) [4]	697,412	Convolution for local features, then a Transformer for relationships across the whole trial	Convolution first, attention after
CTNet (2024) [3]	152,364	An EEGNet-style front end feeding a six-layer Transformer encoder	The encoder design, cut to two layers

Two things stand out when the four are put side by side. Their sizes span more than two orders of magnitude, from about 3,000 to about 700,000 parameters. And their accuracy on unseen people does not follow their size, which is the subject of the next section.

2.3 How published models do on a new person

High accuracy on people who were in the training data says little about a new user. CTNet reports 82.52% on the four-class BCI IV-2a task when the test person's data is in the training set, and 58.64% when it is not, a drop of 23.88 points [3] (Figure 2). On the two-class BCI IV-2b task the drop is 12.22 points.

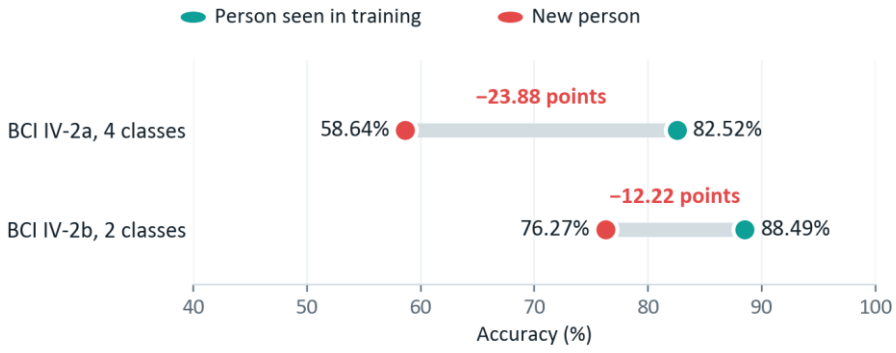


Figure 2. CTNet's published accuracy on people seen in training against people held out, on two datasets.
Values from Zhao et al. [3].

The CTNet authors also re-ran several older models under the same leave-one-subject-out protocol: each person held out in turn, training on the other eight for 600 epochs on the 2–6 s window, with no alignment and no fine-tuning on the held-out person [3]. That table is the only cross-subject comparison of these architectures under one protocol that we could verify, so it is our reference (Table 2). Setting it against model size gives the picture that shaped our design (Figure 3). The largest model, EEG Conformer, has 219 times as many parameters

as EEGNet and is 3.4 points less accurate on a new person. The best of the five is only 3.3 points ahead of EEGNet.

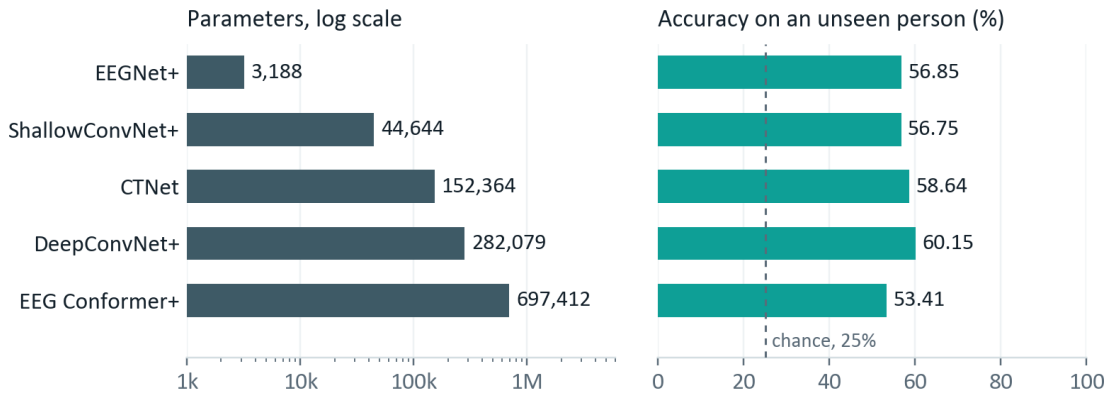


Figure 3. Model size against accuracy on an unseen person, for the models re-run by the CTNet authors.
Accuracy from Zhao et al. [3], where + marks their re-implementations. Parameter counts are for our 875-sample input.

Table 2. Published leave-one-subject-out accuracy (%) on BCI IV-2a, from Zhao et al. [3].
Cohen's kappa for the five rows is 0.4234, 0.4686, 0.4246, 0.3789 and 0.4486.

Model	A01	A02	A03	A04	A05	A06	A07	A08	A09	Mean ± SD
ShallowConvNet+	66.84	46.53	67.53	52.26	34.38	39.76	65.45	71.18	66.84	56.75 ± 13.77
DeepConvNet+	68.58	47.40	78.99	52.26	50.87	41.84	69.44	71.70	60.24	60.15 ± 12.71
EEGNet+	69.79	42.01	79.51	50.87	35.76	37.15	65.80	67.36	63.37	56.85 ± 15.82
EEG Conformer+	68.75	37.33	69.62	43.58	29.51	35.24	58.33	74.48	63.89	53.41 ± 17.08
CTNet	69.27	43.92	79.34	55.38	43.92	36.11	65.10	70.66	64.06	58.64 ± 14.61

Within-subject figures are a different story. They come from different protocols and cannot be ranked against each other (Table 3). The same EEGNet scores 68.67% in the ATCNet authors' evaluation and 77.39% in the CTNet paper's re-implementation.

Table 3. Published within-subject accuracy on BCI IV-2a, with the protocol behind each number.
These are comparable neither with each other nor with Table 2.

Model	Accuracy	Source	How it was measured
CTNet	82.52%	Zhao et al. [3]	Session 1 trains, session 2 tests; 30% of the training data held out for validation; 1,000 epochs
EEG Conformer	78.66%	Song et al. [4], code repository	Hold-out, as stated in the repository
ATCNet	85.38%	Altaheri et al. [2]	Subject-dependent, as stated in the paper
ATCNet	81.10%	ATCNet code repository	Session 1 trains, session 2 tests
EEGNet	68.67%	ATCNet code repository	The same script as the 81.10% above
EEGNet+	77.39%	Zhao et al. [3]	The CTNet paper's subject-specific protocol
EEG Conformer+	77.66%	Zhao et al. [3]	The CTNet paper's subject-specific protocol
FBCSP with LDA	about 68%	Ang et al. [9]	Competition protocol, session 1 trains, session 2 tests; not re-checked by us

3. Research gap and objectives

3.1 The gap

Putting the survey together, we see five problems:

- **New people.** Accuracy falls sharply when the test person was not in the training data, by 23.88 points for CTNet on BCI IV-2a.
- **Calibration.** Most reported results assume labelled data from the target person is available. Working with no calibration, or very little, is still an open problem.
- **Reach against size.** Convolutional models such as EEGNet are small and train reliably, but each filter only sees a short stretch of the trial. Transformer models see the whole trial, but they are much larger and need more data than a dataset like this offers.
- **No fair comparison.** The published numbers for the four models come from different protocols, so they cannot be ranked against each other as they stand.
- **No balance.** None of the four balances accuracy, model size and performance on new people at the same time.

Our project sits in that gap: a small model, built and tested for people it has never seen, compared fairly with the models it borrows from.

3.2 Objectives

1. Build a reproducible preprocessing and evaluation pipeline for BCI Competition IV dataset 2a.
2. Implement EEGNet, ATCNet, CTNet and EEG Conformer, with FBCSP as a classical control, and evaluate all of them under one identical protocol.
3. Design and implement HCT-Net, which combines EEGNet's convolutional front end with windowed Transformer attention.
4. Train and test every model with leave-one-subject-out cross-validation, so every number we report comes from a person the model never saw.
5. Report accuracy, Cohen's kappa, precision, recall and F1 for each person and on average, along with each model's size and running time.
6. State clearly whether the combination helps on new people while staying small, and report the answer whichever way it goes.

4. Dataset

4.1 Why BCI Competition IV 2a

We looked at five public datasets (Table 4) and chose BCI Competition IV dataset 2a [5], [6]. All four papers we build on report results on it, so we can check our implementations against their numbers. It has four classes,

which keeps the task far from trivial, and it is small enough to run full leave-one-subject-out experiments in the time we have.

Table 4. Datasets we considered.

Dataset	People	Channels	Classes	Our assessment
BCI IV-2a	9	22	4	Chosen. All four papers use it, so our baselines can be checked; four classes; small enough for full leave-one-subject-out runs
BCI IV-2b	9	3	2	Only three electrodes and two classes. A useful second check later, but too easy as the main task
High-Gamma Dataset	14	128	4	More people and denser recordings, but 128 channels at 500 Hz is far heavier to process than our schedule allows
PhysioNet Motor Movement/Imagery	109	64	4	Many more people, which would strengthen a cross-subject claim, but none of the four papers uses it, so there is nothing to compare against
SEED	15	62	3	Ruled out: an emotion dataset with no movement labels

4.2 What is in it

Table 5. Properties of BCI Competition IV dataset 2a [6].

Property	Value
People	9, labelled A01 to A09
EEG channels	22, placed over the sensorimotor cortex
Other channels	3 EOG (eye) channels, used to spot artefacts and then dropped
Classes	4: left hand, right hand, both feet, tongue
Sampling rate	250 Hz
Recording filter	0.5 to 100 Hz band-pass, with a 50 Hz notch
Sessions	2 per person, recorded on different days
Runs	6 per session, each with 48 trials (12 per class)
Trials	288 per session, 576 per person, 5,184 in total
Class balance	Exactly balanced, so chance is 25%
Window we use	0.5 to 4.0 s after the cue: 875 samples per channel

4.3 One trial, start to finish

Every trial follows the same timeline [6] (Figure 4). It opens with a fixation cross on the screen and a short beep. Two seconds in, an arrow appears for 1.25 seconds, pointing left, right, down or up to say which movement to

imagine. The person keeps imagining that movement until about the six-second mark, and then rests for a few seconds.

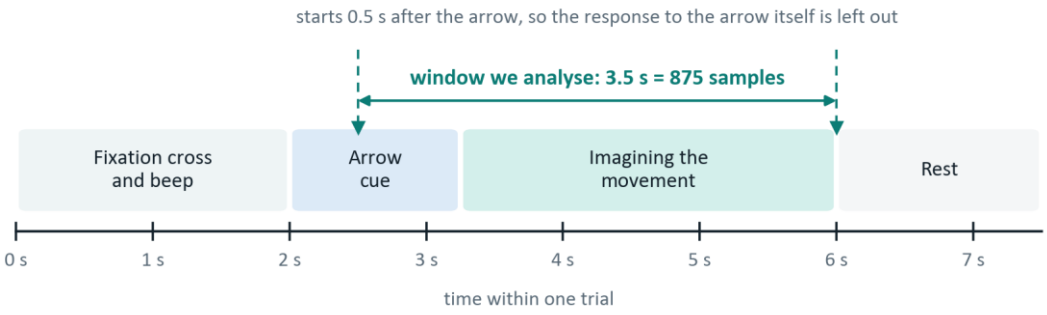


Figure 4. The timeline of one trial and the window we analyse.

We take the window from 0.5 s to 4.0 s after the arrow, which is 2.5 s to 6.0 s into the trial: 3.5 seconds, or 875 samples. We leave out the first half-second on purpose. It mostly holds the brain's visual response to the arrow, and a model could learn to recognise the arrow's direction from that response while knowing nothing about imagined movement. The window ends at six seconds because that is where the imagery stops.

4.4 The four classes

Each class shows up over a different part of the sensorimotor cortex (Figure 5). Hand imagery appears on the opposite side of the head, foot imagery near the midline, and tongue imagery lower down on both sides.

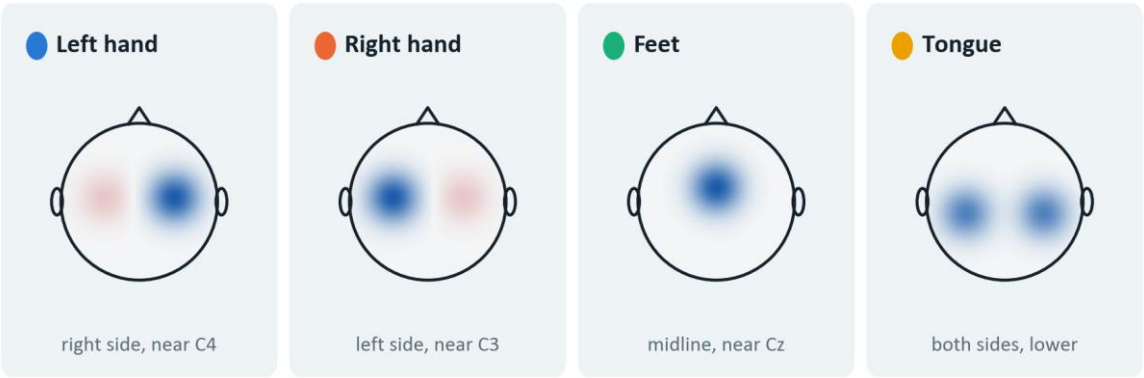


Figure 5. The four imagined movements and where each is expected to appear on the scalp.
Schematic, not recorded data.

4.5 What the data mean for our design

A few properties of the data shape almost every later decision:

- **Feet and tongue are hard to tell apart.** Both appear near the middle of the head and not on opposite sides, so we expect the model to confuse them, and the confusion matrix should show it.
- **People differ a lot.** Some produce clear, steady rhythms and others hardly any, which is sometimes called BCI illiteracy. That is a property of the person, not of the model, and it is why we report results per person.
- **Sessions differ too.** The two sessions were recorded on different days, so even one person's data shifts between them.
- **The dataset is small.** With 5,184 trials in total, regularisation and augmentation are needed, and a large model is at real risk of memorising the training people.

Nine people means nine folds under leave-one-subject-out. That is enough for a mean, a standard deviation and a paired test, but not enough to trust any single fold on its own. Because the classes are balanced, plain accuracy is a fair headline number; we still report macro precision, recall and F1, so that a model that ignores a hard class cannot hide behind the average. And because the dataset is small, we treat a low parameter count as a design requirement.

5. Proposed solution

5.1 Design principles

We fixed five principles before drawing the architecture (Table 6). Each comes from the research gap, and each can be checked.

Table 6. Our design principles.

Principle	What it means	Why	How we check it
P1. New people first	Every design choice is judged by its effect on unseen people	Section 2.3: the 24-point drop is the problem	All headline results use leave-one-subject-out
P2. Small by default	Capacity is added only where a known weakness needs it	Small dataset; modest hardware	Parameter count worked out in Section 8.3
P3. Every part earns its place	Nothing goes in just because a paper used it	A combination without reasons is not a design	Component study, experiment E4
P4. One protocol for all	Baselines and HCT-Net share preprocessing, folds and stopping rules	Published numbers use different protocols	One configuration file per experiment
P5. Report everything	Negative and unclear results get the same detail as positive ones	This is a study, not a demo	Result tables fixed in advance, Section 10.4

5.2 Approaches we considered

We weighed six families of models before settling on a hybrid (Table 7). The judgement is qualitative and based on what the literature reports; we have not measured these ourselves.

Table 7. Model families we considered.

Approach	Why we considered it	Why it is not the main model
FBCSP with LDA [9]	Well understood, fast, stable, and still competitive on small datasets	Its features are fixed in advance and cannot adapt; kept as a classical control
Compact CNN (EEGNet)	Very small and trains reliably; the natural lower bound on size	Each filter sees only a short stretch of the trial; kept as a baseline
Deep CNN (DeepConvNet [8])	More capacity than a compact CNN	Its reported accuracy on this dataset is poor for its size, which suggests overfitting
Recurrent networks (LSTM, GRU)	Built for sequences and can carry information across a trial	Slow, and hard to train stably on short noisy trials; none of the four papers uses them
Pure Transformer on raw EEG	Can relate any two moments of a trial directly	Very large and data-hungry; datasets this size do not support it without heavy augmentation
CNN with windowed attention	Convolution gives cheap spatial filtering, attention gives longer-range structure, and windows keep attention cheap	Chosen. It is acceptable on every criterion, though it is not the best on any single one

5.3 How the pieces fit

The convolutional front end comes from EEGNet. Its depthwise and separable convolutions give spatial filtering similar to common spatial patterns at a very small parameter cost, which serves P2. For attention we use ATCNet's idea of attending within sliding windows instead of across the whole sequence; windows cut the number of parameters and limit how freely attention can fit noise. Putting a Transformer encoder after a convolutional feature map follows CTNet and EEG Conformer, but we use two encoder layers where CTNet uses six. Euclidean alignment goes into preprocessing, because it reduces the differences between people at no parameter cost, which serves P1 and P2 together. Finally, training and testing are subject-independent throughout, which none of the four papers uses as its main setting.

What we borrow, and what is ours

Borrowed: the depthwise and separable convolution block (EEGNet), sliding-window attention (ATCNet), convolution followed by a Transformer (CTNet and EEG Conformer), Euclidean alignment (He and Wu [7]), and segmentation-and-reconstruction augmentation (EEG Conformer and CTNet).

Ours: combining these into one small model, cutting attention depth to suit the data, applying the same subject-independent procedure to every model, and a component study that measures what each part adds on unseen people.

6. System architecture

6.1 From raw EEG to a prediction

The system is a chain of stages from the recording to a predicted class (Figure 6). Only two of them, the CNN front end and the windowed attention, have trainable weights; the rest is fixed signal processing or the final output layer (Table 8).

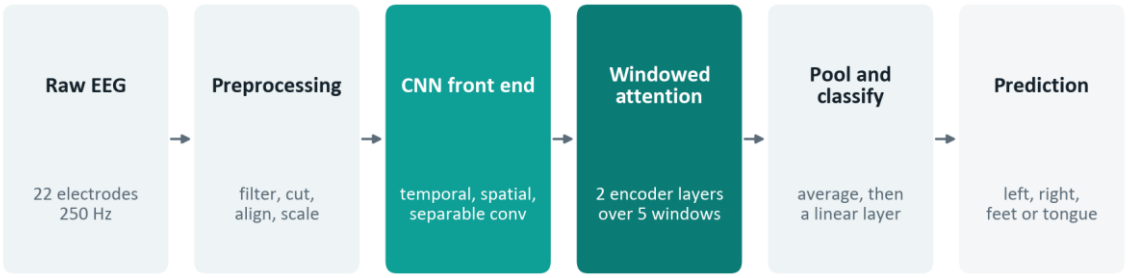


Figure 6. The system end to end.
The two coloured stages are the learned part of HCT-Net.

Table 8. What each stage does.

Stage	What it does	Why it is there
Raw EEG	One trial from 22 electrodes at 250 Hz	The input, as the dataset provides it
Preprocessing	Filtering, cutting trials, alignment and scaling	Removes noise outside the bands we need and puts everyone on a common scale
CNN front end	Temporal, depthwise spatial and separable convolution	Learns frequency filters and electrode combinations cheaply
Windowed attention	Multi-head self-attention within overlapping windows	Relates different moments of the trial, which a short convolution cannot
Pool and classify	Averages over windows and time, then a linear layer with softmax	Turns the features into one probability per class
Prediction	Left hand, right hand, feet or tongue	The output

6.2 Software modules

We split the code so that each module has one job, an error in one stage cannot quietly spread to another, and the expensive preprocessing runs once instead of once per model (Table 9).

Table 9. Software modules.

Module	Job	Takes in	Produces
data/	Load the dataset, pick channels, define the folds	Raw recordings	Raw epochs, fold definitions
preprocessing/	Filter, cut, reject, align, scale, augment and cache	Raw epochs	Cached arrays on disk
models/	Every architecture, baselines included	Configuration	Models with a common interface
training/	One training run for one fold	Cached arrays, model, configuration	Trained weights, training log
evaluation/	The leave-one-subject-out loop, metrics and statistical tests	Trained weights, cached arrays	Per-fold metrics, test results
analysis/	Every figure and table in the final report	Per-fold metrics	Figures and tables
configs/	One file per experiment	—	YAML files
results/	Metrics, logs and checkpoints	—	Files under version control, except checkpoints

6.3 What is learned and what is fixed

The split matters when we compare parameter counts with the baselines (Table 10). Alignment reduces the differences between people without adding a single parameter, which is why it lives in preprocessing and not inside the network.

Table 10. Trainable and fixed parts of the system.

Component	Trainable?	Notes
Band-pass filter	No	Fixed fourth-order Butterworth
Cutting trials and rejecting artefacts	No	Deterministic, given the thresholds
Euclidean alignment	No	One matrix per person and session, from their own unlabelled trials
Standardisation	No	Statistics from the training people
Convolutional block	Yes	3,424 parameters
Windowed attention block	Yes	17,440 parameters, including the positional encoding
Classifier	Yes	132 parameters

7. Preprocessing

Every model gets exactly the same preprocessing, so any difference in results comes from the models and not from how the data was handled.

7.1 The eight steps

The steps run in the order shown in Table 11.

Table 11. The preprocessing pipeline.
Every setting is fixed here, so the code needs no further decisions.

#	Step	Settings	Why
1	Select channels	Keep the 22 EEG channels; drop the 3 EOG channels after using them to check for artefacts	The eye channels are not model inputs, but they help find contaminated trials
2	Band-pass filter	4 to 38 Hz, fourth-order Butterworth, run forwards and backwards	Keeps the mu and beta rhythms and removes slow drift and mains noise; running both ways means no phase shift
3	Cut trials	0.5 to 4.0 s after the cue, 875 samples	Leaves out the visual response to the cue and stops when the imagery stops
4	Reject artefacts	Drop a trial if any channel swings more than 100 µV peak to peak	Removes trials dominated by blinks or movement; we record the rejection rate for each person
5	Euclidean alignment	Per person and per session; see Section 7.2	Reduces the differences between people before the network sees the data, at no parameter cost
6	Standardise	Per channel, with the mean and standard deviation of the training people only	Puts channels on a comparable scale without using anything from the test person
7	Augment	Segmentation and reconstruction, training data only; see Section 7.3	More training trials from a small dataset
8	Cache	Save arrays and labels to disk	Preprocessing runs once, which is what makes the schedule possible

7.2 Euclidean alignment

We expect this step to do most of the work against the new-person problem, so it deserves a proper explanation [7].

Any EEG trial can be summarised by its spatial covariance matrix: a 22 × 22 table showing how strongly each pair of electrodes moves together. That matrix reflects how activity inside the brain is mixed on its way to the scalp, and the mixing depends on the person: the thickness and conductivity of the skull, the folding of the cortex, exactly where the cap sits. None of this has anything to do with which movement the person is imagining, yet all of it shapes the recording. It is the main reason a model trained on some people transfers badly to someone new.

Alignment removes most of that person-specific mixing in two steps. First, we average the covariance matrices of all N trials recorded from one person in one session, where each trial is a 22 × 875 matrix (Equation 1):

$$\overline{R} = \frac{1}{N}\sum_{i=1}^{N} X_i X_i^T \tag{1}$$

With the four classes equally represented, this average describes how that person's signals are mixed in general, with the class-specific detail averaged out. Second, we multiply each trial by the inverse square root of the average (Equation 2):

$$\tilde{X}_i = \overline{R}^{-1/2} X_i \tag{2}$$

Afterwards, the average covariance of the aligned trials is the identity matrix (Figure 7).

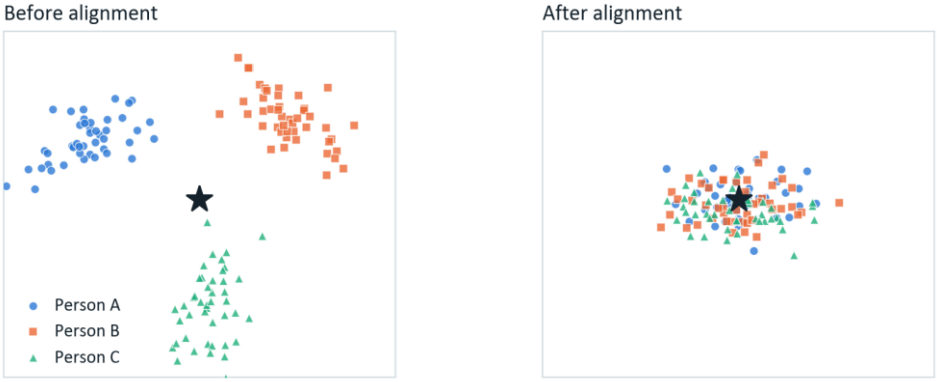


Figure 7. What alignment does, sketched for three people.

Before, each person's trials sit in their own region. After, every person's average covariance is moved to the identity (the star), so the model sees everyone in the same frame. Illustration, not measured data.

So every person reaches the network with the same average spatial statistics. The model no longer has to learn that one person's signals are larger at a certain electrode, or that two electrodes happen to be more correlated in one head than in another. What is left is the variation around the shared average, and that is where the class information lies.

We compute the inverse square root through an eigendecomposition: decompose the average matrix, raise each eigenvalue to the power −½, and put the matrix back together. If two electrodes are nearly identical, for example because a channel was flat, an eigenvalue can be close to zero and its inverse square root enormous, so we clamp small eigenvalues to a small positive floor first.

Two properties make alignment a good fit for this project. It needs no labels, only the trials themselves, so it works for a new user who has not given any labelled data. And it adds no trainable parameters: the whole cost is one eigendecomposition of a 22 × 22 matrix per person and session. We will check it in code as well. After

alignment, each person's average covariance must equal the identity within a small tolerance; that check is a unit test and the completion condition for milestone M2.

7.3 Data augmentation

Each training fold has about 4,032 trials (seven people × 576), which is small for a neural network, so we augment. We use segmentation and reconstruction, the method used by EEG Conformer and CTNet [3], [4]. Each trial is cut into eight equal pieces along time. A new trial is then built by taking the first piece from one real trial, the second from another, and so on, with every donor drawn from trials of the same class. The result never happened in a recording, but every piece of it is real EEG from that class.

This works because motor imagery is a sustained task: the person keeps imagining throughout the window, so one stretch of a trial carries the same kind of information as another. Each piece keeps all 22 channels together, so the spatial structure is untouched. What gets broken is the continuity at the joins and the link between distant parts of a trial. That is why the number of pieces matters. A few long pieces give trials close to real ones but little variety; many short pieces give variety at the cost of signals that no longer look like EEG. Eight pieces of about 0.44 s each is what the papers settle on, and we will confirm it on the validation person, never on the test person.

Augmentation is only ever applied to training data. Validation and test trials are always real recordings.

7.4 Keeping the test person out

Leakage rules, with no exceptions

Standardisation statistics come from the training people only, never from the validation or test person.

Augmentation touches training data only.

Every hyperparameter, including the early stopping point, is chosen on the validation person only.

The test person contributes unlabelled trials to their own alignment and nothing else. A real user could provide the same, since unlabelled trials can be recorded before any labels exist.

Breaking any of these rules would make the cross-subject result meaningless, so any departure is reported as a limitation and never quietly fixed.

7.5 Checks before any training

Before training anything, we look at the data. We want to confirm that the dataset is what its description says, that the pipeline behaves, and how much people differ. Each check has an expected result, so a surprise gets noticed instead of accepted (Table 12).

Table 12. Checks on the data before any model is trained.

Check	What it tells us	What we expect
Trials and class balance per person	Whether any person or class is short after rejection	576 trials per person and 144 per class, before rejection
Artefact rejection rate per person	Differences in data quality	Under about 15% for most people
Power spectrum per channel and class	Whether mu and beta activity are present	Peaks near 10 Hz and 20 Hz over the motor channels
Time–frequency maps at C3, Cz and C4	Whether desynchronisation follows the cue	Mu power drops from about 0.5 s after the cue
Average scalp maps per class	Whether hand imagery shows up on the opposite side	Left hand strongest near C4, right hand near C3
Covariance distance between people	How far apart people are, before and after alignment	Much smaller distances after alignment
2-D projection of trial features	Whether trials group by class or by person	Grouped by person before alignment, much less after

If a check disagrees with its expectation, we find out why before training anything. An error in the data that slips through here would affect every later result and would be very hard to trace afterwards. We keep the figures from these checks for the final report, because they show the pipeline was verified and not just assumed to work.

8. HCT-Net

8.1 Constraints

We set the constraints before designing the network (Table 13).

Table 13. Constraints fixed before the design.

Constraint	Value	Reason
Trainable parameters	Under 50,000	A ceiling near ShallowConvNet's 44,644 and far below ATCNet's 113,732; a small model has less room to memorise individual people
Attention span	Windowed, not global	Global attention over 27 time steps is affordable but unconstrained; windows add a locality prior that suits short trials
Encoder depth	2 layers	CTNet uses 6; with under 6,000 trials, extra depth is more likely to fit the training people than what they share
Input length	875 samples	Set by the analysis window, Section 4.3
Output	4 classes	Set by the dataset

8.2 Architecture

Figure 8 shows HCT-Net with the shape of the data after each block, and Table 14 gives the exact settings for 22 channels and 875 samples.



Figure 8. The HCT-Net architecture.
Shapes on the right are for BCI IV-2a; the tags name the paper each block is adapted from.

Table 14. Layer-by-layer specification of HCT-Net.

#	Layer	Settings	Output shape
1	Temporal convolution	16 filters, kernel (1, 64), same padding, no bias	16 × 22 × 875
2	Batch normalisation	—	16 × 22 × 875
3	Depthwise spatial convolution	Depth multiplier 2, kernel (22, 1), max-norm 1.0, no bias	32 × 1 × 875
4	Batch normalisation, ELU	—	32 × 1 × 875
5	Average pooling, dropout	Pool (1, 4), dropout 0.25	32 × 1 × 218
6	Separable convolution	32 filters, kernel (1, 16), depthwise then pointwise	32 × 1 × 218
7	Batch normalisation, ELU	—	32 × 1 × 218
8	Average pooling, dropout	Pool (1, 8), dropout 0.25	32 × 1 × 27
9	Window segmentation	5 overlapping windows of length 11, stride 4	5 × 32 × 11
10	Positional encoding	Learned, shape (11, 32), added to each window	5 × 32 × 11
11	Transformer encoder	2 layers, 2 heads, model width 32, feed-forward 64, GELU	5 × 32 × 11
12	Pooling	Mean over windows, then over time	32
13	Classifier	Fully connected, 32 → 4, softmax	4

8.3 Parameter budget

We worked out the parameter count by hand, so the claim that the model is small can be checked before any code exists (Table 15). The convolutions have no bias terms, as in EEGNet, and the feed-forward layer inside the encoder expands by a factor of two.

Table 15. The parameter budget, worked out by hand.

Component	Calculation	Parameters
Temporal convolution	16 × 1 × 1 × 64	1,024
Batch normalisation 1	2 × 16	32
Depthwise spatial convolution	16 × 2 × 22 × 1	704
Batch normalisation 2	2 × 32	64
Separable convolution, depthwise	32 × 1 × 16	512
Separable convolution, pointwise	32 × 32	1,024
Batch normalisation 3	2 × 32	64
Convolutional block		**3,424**
Attention projections, per layer	4 × (32 × 32 + 32)	4,224
Layer normalisation, per layer	2 × 2 × 32	128
Feed-forward, per layer	(32 × 64 + 64) + (64 × 32 + 32)	4,192
One encoder layer	4,224 + 128 + 4,192	**8,544**
Encoder, 2 layers	2 × 8,544	**17,088**
Learned positional encoding	11 × 32	352
Classifier	32 × 4 + 4	132
Total	3,424 + 17,088 + 352 + 132	**20,996**

In context, HCT-Net is about a fifth of the size of ATCNet and a thirty-third of EEG Conformer, but 6.6 times the size of EEGNet (Table 16). That last comparison matters most. If HCT-Net cannot beat EEGNet on new people, its attention is not paying for itself.

Table 16. Parameter counts of HCT-Net and the published models.

Counts are for 22 channels and four classes, from the braindecode 1.8.1 reference implementations, at our 875-sample input and at the 1,125 samples used by the ATCNet reference code. The ratio compares like with like, at 875 samples. Some models size their classifier from the input, so their count depends on the window length; the 2,548 often quoted for EEGNet corresponds to a window of about 2.2 s.

Model	At 875 samples	At 1,125 samples	Relative to HCT-Net
EEGNet	3,188	3,700	0.15 ×
EEG-TCNet	4,304	4,304	0.20 ×
HCT-Net (ours)	**20,996**	—	**1.00 ×**
ShallowConvNet	44,644	47,364	2.13 ×
ATCNet	113,732	113,732	5.42 ×
CTNet	152,364	153,004	7.26 ×
DeepConvNet	282,079	284,479	13.43 ×
EEG Conformer	697,412	871,492	33.22 ×

8.4 Following one trial through the network

A preprocessed trial enters as 22 channels by 875 samples. The first layer convolves along time only, with sixteen filters that are each 64 samples long: about a quarter of a second, or two to three cycles of the mu

rhythm. Each filter ends up responding to its own frequency band. Because the filters are learned, the network picks its own bands instead of being limited to the mu and beta ranges we would have chosen. The trial is now sixteen filtered copies of itself, each still with all 22 channels.

Next, a depthwise convolution works across the channels. Its kernel spans all 22 electrodes and a single time sample, so at each moment it forms a weighted mix of electrodes. This is the idea behind common spatial patterns, except that it is learned together with everything else. With a depth multiplier of two, each frequency band gets two spatial filters, 32 feature maps in total, and the channel dimension collapses to one. A max-norm limit on these weights stops any single electrode mix from growing without bound.

Two pooling steps then shorten the time axis, first by four (875 to 218 steps) and then by eight (218 to 27). Shrinking time this hard is safe because the information in a mu rhythm lies in how its power changes over hundreds of milliseconds, not in where individual peaks fall. Between the two pooling steps, a separable convolution first summarises each feature map over time and then mixes the maps, which does the job of a full convolution for a fraction of the parameters.

What leaves the convolutional block is a sequence of 27 steps with 32 features each. Instead of letting every step attend to every other, we cut the sequence into five windows of eleven steps, one starting every four steps, so neighbouring windows overlap. The overlap matters: without it, a window boundary could split an informative stretch of signal between two windows that never see each other. A learned positional encoding is added to each window, because self-attention on its own ignores order. Inside each window, two attention heads work out how much each step should draw on every other, using scaled dot-product attention [10] (Equation 3):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V \tag{3}$$

Here Q, K and V are learned projections of the window's features, and d = 16 is the width of each head. A small feed-forward network then transforms the result, and this repeats over two encoder layers. A feature at one moment can now be read in the light of what happens at another moment in the same window, which a convolution cannot do beyond the length of its kernel.

Finally the five window outputs are averaged, and so is the time axis inside them, which gives one vector of 32 numbers for the whole trial. A linear layer maps it to four scores, a softmax turns those into probabilities, and the predicted class is the most probable one.

8.5 Why two encoder layers, not six

CTNet uses six encoder layers and reports strong within-subject accuracy. We use two. Each fold trains on only about 4,032 trials, and six layers at our width would triple the attention parameters, from 17,088 to 51,264, which on its own breaks the 50,000 budget before the convolutional block is counted.

There is a better reason than the budget. More attention capacity gives the model more room to fit what is specific to the training people, and that does not carry over to a new person. The gap in Figure 2 is exactly what fitting person-specific detail looks like. Cutting capacity is therefore a sensible response to the problem and not just a concession to hardware. Whether it is the right response is an empirical question, and experiment E5 tests it directly.

9. Training and evaluation

9.1 Training set-up

Every model is trained with the same settings (Table 17). These are starting values. If we change one during tuning, we change it for every model and record it in that experiment's configuration file.

Table 17. Training settings.

Setting	Value	Reason
Loss	Cross-entropy, label smoothing 0.1	Smoothing curbs overconfidence, which matters when a pattern has been seen only a few times
Optimiser	Adam, learning rate 0.001	Adam is the optimiser all four papers use; changing it would make comparisons harder to read
Learning-rate schedule	Cosine decay over the maximum number of epochs	Big steps early and small steps late, without a hand-tuned schedule
Batch size	64	Fits easily in GPU memory and gives stable batch-norm statistics
Maximum epochs	500	A generous ceiling; early stopping should end training first
Early stopping	Patience of 50 epochs on validation-person accuracy	Stops before the model starts fitting the training people specifically
Dropout	0.25 after each pooling layer	The value EEGNet's authors use for cross-subject classification; they use 0.5 within subject
Weight constraint	Max-norm 1.0 on the depthwise convolution	From EEGNet: stops any spatial filter from dominating
Weight decay	0.0001	Mild extra regularisation, the same for every model
Augmentation	Segmentation and reconstruction, training data only	More training trials; never applied to validation or test data
Random seeds	Three per configuration, results averaged	One seed can produce a difference between models that is only initialisation noise
Mixed precision	On where supported	Cuts training time by roughly a third, which matters given the number of runs

Each epoch shuffles the training trials into batches of 64. For each batch the model scores the trials, we compute the smoothed cross-entropy loss, and the gradients update the weights; right after each update we re-apply the max-norm limit to the spatial filters. At the end of every epoch we switch to evaluation mode, which turns dropout off and freezes the batch-norm statistics, and measure accuracy on the validation person. If it is the best so far, we save the weights and reset the patience counter. If not, the counter goes up, and training stops when it reaches 50. We keep the best checkpoint, not the last one.

Every run fixes the seeds of Python, NumPy and PyTorch and records them with its results, and we turn on deterministic algorithms where PyTorch offers them. Bit-for-bit reproducibility on a GPU is not always possible, so we do not claim it. Instead we check that re-running a configuration with the same seed reproduces the reported accuracy within a small tolerance, and we report that tolerance.

If time allows, we will also try one extra variant: a branch that tries to predict which person a trial came from, with its gradient reversed before it reaches the feature extractor, so that the network is pushed to drop person-specific information. This is an extension, not part of the main model, because it adds a weighting that would itself need tuning. If we do not get to it, it goes into future work and we say so.

9.2 Leave-one-subject-out

Every headline number comes from leave-one-subject-out cross-validation. With nine people there are nine folds. In each fold one person is the test person, the next one in order is the validation person, and the other seven train the model (Figure 9). The roles rotate, so everyone is the test person exactly once. We report the mean and standard deviation across the nine folds, and the result for each person.

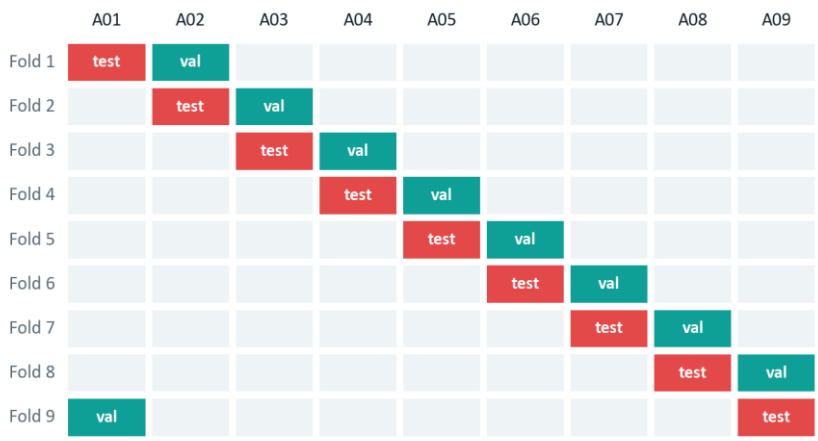


Figure 9. The leave-one-subject-out fold structure.
In every fold one person is tested (red), the next one validates (teal), and the other seven train.

Each fold starts from a freshly built model with reset seeds; carrying weights over between folds would mean the model had already seen the person it is about to be tested on. The test person's trials are never augmented and play no part in choosing weights or hyperparameters. The loop is written once and shared by every model, so no model can end up under a slightly different protocol by accident.

9.3 Why a validation person and not a random split

We hold out a whole person for validation instead of a random slice of the training people's trials. If validation trials came from people who are also in the training set, early stopping and tuning would favour fitting familiar people, which is the opposite of what we want. Holding out a person costs one person's worth of training data in every fold, and we accept that cost.

9.4 A within-subject check

As a sanity check, we also train and test every model within each person, from session one to session two. This is not a headline result. If a baseline's within-subject accuracy is far from its published value, something in our implementation or preprocessing is wrong, and the cross-subject numbers cannot be trusted either. Reporting both protocols also shows how much accuracy each model loses on a new person.

9.5 Metrics

Every model is scored with the same set of metrics (Table 18).

Table 18. Evaluation metrics.

Metric	Definition	Why we use it
Accuracy	Share of trials classified correctly	The headline number, as a mean and standard deviation over nine folds; fair here because the classes are balanced
Cohen's kappa	Agreement corrected for chance	Standard in BCI work and comparable with published results; 0 means chance and 1 means perfect
Precision (macro)	Of the trials predicted as a class, the share that truly belong to it	Shows whether a class is over-predicted
Recall (macro)	Of the trials in a class, the share the model finds	Shows whether a class is being missed
F1 (macro)	Harmonic mean of precision and recall	One number for per-class balance, so a model that ignores a hard class cannot hide behind the average
Confusion matrix	Counts of predicted against true classes	Shows which classes get mixed up, feet and tongue in particular
Parameter count	Number of trainable parameters	For the size comparison, and to check the budget in Table 15
Training time per fold	Wall-clock time of one training run	Whether the schedule is feasible
Inference time per trial	Wall-clock time of one forward pass	Whether the model could run in real time

Results are always reported per person as well as on average. Cross-subject decoders often fail badly on one or two people, and a model with a slightly lower mean but a smaller spread across people may be the better choice in practice. That difference disappears if only the average is shown.

9.6 Deciding whether a difference is real

With nine folds, comparing two means is not enough. We use this procedure:

1. For each pair of models, take the nine paired per-person accuracies.
2. Apply the two-sided Wilcoxon signed-rank test [15] at a significance level of 0.05. It assumes no normal distribution, and it is paired, because every model is tested on the same nine people.
3. When HCT-Net is compared with all five baselines, apply the Holm–Bonferroni correction [14] across the five comparisons and report both the corrected and uncorrected values.
4. Report an effect size next to every p-value: with nine people a non-significant result may still matter in practice, and a significant one may not.
5. Call an unclear comparison inconclusive instead of describing it as a trend.

10. Experiments

10.1 What varies and what stays fixed

The model is the one thing we vary. Everything else is held fixed, so that any difference can be traced to its cause (Table 19).

Table 19. Variables and controls.

Role	Item	How it is handled
Varied	Model architecture	Six models, plus five component-study variants
Varied	Encoder depth (E5 only)	Two layers against six
Measured	Accuracy, kappa, precision, recall, F1, size and time	On every fold
Fixed	Dataset and preprocessing	The same cached arrays for every model
Fixed	Folds	The same nine train, validation and test splits
Fixed	Optimiser, schedule, batch size, stopping rule	Identical, as in Table 17
Fixed	Random seeds	The same three seeds for every configuration
Nuisance	Differences in signal quality between people	Handled by pairing: the Wilcoxon test compares models on the same people
Nuisance	GPU non-determinism	Reduced by seeding; what remains is measured through the three seeds

10.2 The experiments

Six experiments make up the study, and each one answers a single question (Table 20).

Table 20. The planned experiments.
That is 153 training runs for one seed, and every configuration is repeated with three seeds.

ID	Experiment	Set-up	Question	Runs
E1	Baselines, within subject	Each baseline, session 1 to session 2	Do our implementations reproduce the papers?	5 × 9 = 45
E2	Baselines, unseen people	Each baseline under leave-one-subject-out	How do they compare under one protocol?	5 × 9 = 45
E3	HCT-Net, unseen people	HCT-Net under leave-one-subject-out	How does our model do on new people?	9
E4	Component study	Five variants, each with one part removed or changed	Which parts matter on new people?	5 × 9 = 45
E5	Attention depth	Two encoder layers against six	Is the reasoning in Section 8.5 right?	9
E6	Size and speed	Parameter count, training and inference time	Is HCT-Net really lighter?	During E2 and E3

10.3 The component study

E4 removes or changes one part of HCT-Net at a time and repeats the full evaluation (Table 21). V0 against V1 isolates attention, V0 against V3 isolates alignment, and V0 against V5 gives the total contribution of everything added on top of the convolutional core. This is what separates a designed model from an assembled one.

Without it, a combination of published parts is only a combination; with it, we can say which parts mattered on new people.

Table 21. Component-study variants.

Variant	Convolution	Windowed attention	Alignment	Augmentation
V0 full model	✓	✓	✓	✓
V1 no attention	✓	removed	✓	✓
V2 global attention	✓	made global	✓	✓
V3 no alignment	✓	✓	removed	✓
V4 no augmentation	✓	✓	✓	removed
V5 convolution only	✓	removed	removed	removed

10.4 Result tables, fixed in advance

We fix the format of the main results table now, before any result exists, so that results cannot be presented selectively afterwards (Table 22). A second table will give each model's accuracy, kappa, precision, recall, F1, parameter count and training time per fold, and a third will list each component-study variant with its change from V0 and whether that change is significant. The interpretation is written only after the numbers are in.

Table 22. Template for the main result: accuracy (%) on each unseen person.

Model	A01	A02	…	A09	Mean	SD	Kappa	p vs ours	Holm p
FBCSP with LDA									
EEGNet									
ATCNet									
EEG Conformer									
CTNet									
HCT-Net (ours)								—	—

11. Implementation plan

11.1 Tools

We will build everything with the tools in Table 23.

Table 23. Tools and frameworks.

Area	Tool	Used for
Language	Python 3.10	Everything
Deep learning	PyTorch 2.x with CUDA	Models, automatic differentiation, training
EEG processing	MNE-Python 1.6 or later [11]	Loading, filtering, epoching, scalp maps
Data access	MOABB 1.0 or later [12]	Downloading and handling the dataset
Reference models	braindecode 0.8 or later	Checked implementations of EEGNet, ShallowConvNet and related models
Classical baseline	scikit-learn 1.4, pyRiemann 0.6	FBCSP with LDA
Numerics	NumPy 1.26, SciPy 1.11	Filtering, alignment matrices, statistical tests
Figures	Matplotlib 3.8, Seaborn 0.13	Figures, confusion matrices, scalp maps
Experiment tracking	TensorBoard or Weights & Biases	Logging every run with per-fold metrics
Configuration	YAML	One file per experiment
Version control	Git and GitHub	Code, configurations and result tables

We use published reference implementations for the baselines wherever they exist. If we wrote a baseline from scratch and it missed its published accuracy, we could not tell a bug from a genuine difference in evaluation, and the build has no time to settle that question for four architectures. We also pin the environment in a lock file, store each configuration with the git commit and seeds of every run, cache preprocessed data under a name that includes a hash of the preprocessing settings (so a settings change can never silently reuse stale data), and checkpoint every epoch, so a dropped session on a hosted GPU costs one epoch and not one fold. Unit tests cover the tensor shapes of every model and the alignment check, and they run before any experiment.

11.2 Order of work

The build has eighteen days. We build and verify the data pipeline and the baselines before writing HCT-Net. A result from a new model on an unverified pipeline cannot be interpreted: if accuracy is low, we could not tell whether the model is weak or the data is being handled wrongly, and finding out late would leave no time to fix it.

The stages also differ in how far they can be squeezed. Writing code can be sped up by working longer; training runs take as long as they take. So the compute-heavy stages come last, with as few open questions in front of them as possible, and the coding stages, which can absorb pressure, come first. Three rules follow from this, and we will not bend them:

1. No model trains on cached data until the alignment check in Section 7.2 passes and the data checks in Table 12 match their expectations.
2. HCT-Net is not written until at least two baselines reproduce their published within-subject accuracy.
3. No leave-one-subject-out experiment starts until the fold definitions have been checked, since an error there would spoil every run after it.

A useful consequence is that writing HCT-Net depends only on preprocessing being finished, not on the baselines. We can write and unit-test it while baseline training runs occupy the GPU, and that overlap is the main slack in a tight schedule.

11.3 Timeline

Figure 10 shows the order of the stages and how long each one takes.

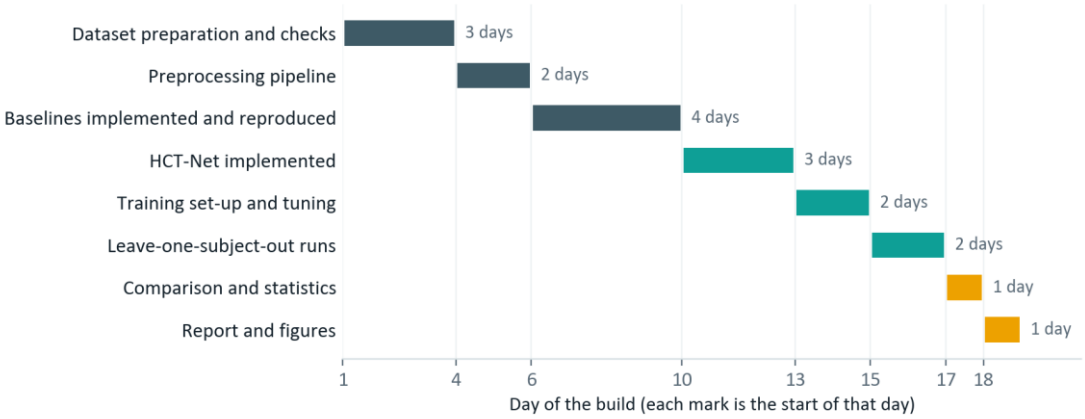


Figure 10. The order and relative length of the implementation stages over the eighteen days.

11.4 Milestones

Each milestone is a condition that is either met or not, never an amount of work done, so it cannot be ticked off while a problem is still hiding inside it (Table 24).

Table 24. Milestones and what counts as done.

ID	Milestone	Done when
M1	Dataset ready and verified	All nine people load; trial counts, class balance, channel count and sampling rate match Table 5; the data-check figures are made and reviewed against Table 12
M2	Preprocessing complete	Filtering, cutting, rejection, alignment and caching work; the average covariance after alignment matches the identity within tolerance; rejection rates are recorded per person; arrays are cached once and reused
M3	Baselines reproduced	All five baselines run end to end; within-subject accuracy lands within about three points of the published value, or the difference is explained
M4	HCT-Net implemented	Shapes match Table 14 at every layer; the parameter count is within 5% of 20,996; the model reaches near-zero loss when deliberately overfitted on one batch
M5	Training set-up fixed	Hyperparameters chosen on the validation person only; configuration files committed; a smoke test over two folds runs cleanly; resuming from a checkpoint is confirmed
M6	Cross-subject experiments done	E2 and E3 finished for every model, all nine folds, three seeds each; per-fold metrics saved
M7	Component study and analysis done	E4 and E5 finished; metrics, confusion matrices and statistical tests computed; the result tables of Section 10.4 filled in
M8	Report ready for the next review	All tables and figures made; findings written up, negative or inconclusive ones included; repository tidy and documented

The milestones are not the same size. M1 and M2 are short but block everything after them, so a delay there costs the most. M3 is the longest coding block and the one most likely to surprise us, since reproducing four published models is where discrepancies show up. M6 is the longest block of compute and the least flexible, because the number of runs is set by the experiment design.

12. Expected outcomes, risks and contribution

12.1 What we expect

These are expectations. No experiment has been run yet, and nothing below is a result (Table 25).

Table 25. Expected outcomes.

We expect	How we will check
A working implementation of five baselines and HCT-Net on one shared pipeline	Code in a version-controlled repository, with recorded configurations
A fair comparison of every model under one leave-one-subject-out protocol	The template in Table 22, per person and on average
Accuracy on new people that holds up against the baselines	Mean accuracy and kappa over nine folds, with a paired test
Fewer parameters than the attention-based baselines	A direct count against Table 15
Evidence of which parts affect accuracy on new people	The component study, E4
A clear statement of whether the combination helps	Reported whether the answer is yes, no or unclear

HCT-Net may well not beat the baselines on new people. If that happens, the component study will still show which parts of the design were responsible, and that is a useful finding in its own right. We built the plan so that a negative result still teaches us something, which is why the component study is a core experiment and not an optional extra.

12.2 Risks and how we will respond

Table 26 lists what could go wrong and what we will do about each problem.

Table 26. Risks and the responses we have planned.

Risk	Why it might happen	What we will do
Too little training data	5,184 trials in total, about 4,032 for training in each fold	Augmentation, dropout, weight limits, early stopping and a small model
Overfitting to the training people	The model can learn person-specific detail that does not transfer	Validate on a held-out person, not a random split; keep capacity low by design
Baselines miss their published accuracy	Preprocessing and evaluation differ between papers	Use the authors' reference implementations; report both numbers and explain the gap
Large spread across folds	Only nine people, some with weak rhythms	Report results per person with the standard deviation; use a paired test, not just means
Not enough GPU time	153 runs per seed, each repeated with three seeds	Cache preprocessed data, use mixed precision, lower the epoch ceiling if needed, checkpoint every epoch
Sessions cut off on hosted GPUs	Time limits on free tiers	Checkpoints in persistent storage, so an interruption costs one epoch and not one fold
HCT-Net does not beat the baselines	The combination may not help on this dataset	Report it plainly and use the component study to explain it
Unstable attention during training	A small model width and short sequences can make attention collapse	Watch attention entropy in the smoke test; fall back to one encoder layer if needed, and record the change

12.3 What is new, and what is not

None of HCT-Net's parts is new. The depthwise and separable convolutions come from EEGNet, sliding-window attention from ATCNet, convolution followed by a Transformer from CTNet and EEG Conformer, Euclidean

alignment from the transfer-learning work of He and Wu [7], and segmentation-and-reconstruction augmentation from EEG Conformer and CTNet. Leave-one-subject-out evaluation is a standard protocol. We do not claim any of these as our own.

What the project adds:

- A comparison of EEGNet, ATCNet, CTNet and EEG Conformer under one identical subject-independent protocol. Their published numbers come from different settings and cannot be ranked as they stand.
- A specific small combination of these parts, at 20,996 parameters, with attention depth cut to suit the dataset, built and judged for new people and not for familiar ones.
- A component study measuring how much each part adds to accuracy on new people, which the four papers do not report.
- Accuracy set against parameter count under a single protocol, so we can judge whether attention-based models justify their size on a dataset like this.

So the contribution is a careful comparison and a concrete small design, not a new architecture. We say this plainly, and we make no claim about performance before the experiments are run.

13. Future work

- Run the same evaluation on BCI Competition IV dataset 2b and on larger collections such as PhysioNet motor imagery, to see whether the findings hold with more people and different electrode layouts.
- Measure how many labelled trials a new user needs to reach a given accuracy, which would put a number on the calibration effort a real system needs.
- Try more systematic ways of reducing differences between people, including the adversarial variant in Section 9.1 and contrastive methods.
- Use interpretability methods such as class activation maps to check that the model relies on the expected motor regions and not on artefacts.
- Test the model online, with trials arriving continuously and a decision needed within a fixed delay.
- Shrink the model further with quantisation or pruning, with a view to running it on wearable hardware.

References

[1] V. J. Lawhern, A. J. Solon, N. R. Waytowich, S. M. Gordon, C. P. Hung and B. J. Lance, “EEGNet: A Compact Convolutional Neural Network for EEG-based Brain–Computer Interfaces,” Journal of Neural Engineering, vol. 15, no. 5, 056013, 2018.

[2] H. Altaheri, G. Muhammad and M. Alsulaiman, “Physics-Informed Attention Temporal Convolutional Network for EEG-Based Motor Imagery Classification,” IEEE Transactions on Industrial Informatics, vol. 19, no. 2, pp. 2249–2258, 2023.

[3] W. Zhao, X. Jiang, B. Zhang, S. Xiao and S. Weng, “CTNet: A Convolutional Transformer Network for EEG-Based Motor Imagery Classification,” Scientific Reports, vol. 14, 20237, 2024.

[4] Y. Song, Q. Zheng, B. Liu and X. Gao, “EEG Conformer: Convolutional Transformer for EEG Decoding and Visualization,” IEEE Transactions on Neural Systems and Rehabilitation Engineering, vol. 31, pp. 710–719, 2023.

[5] M. Tangermann, K.-R. Müller, A. Aertsen et al., “Review of the BCI Competition IV,” Frontiers in Neuroscience, vol. 6, 55, 2012.

[6] C. Brunner, R. Leeb, G. Müller-Putz, A. Schlögl and G. Pfurtscheller, “BCI Competition 2008 – Graz Data Set A,” Institute for Knowledge Discovery, Graz University of Technology, 2008.

[7] H. He and D. Wu, “Transfer Learning for Brain–Computer Interfaces: A Euclidean Space Data Alignment Approach,” IEEE Transactions on Biomedical Engineering, vol. 67, no. 2, pp. 399–410, 2020.

[8] R. T. Schirrmeister, J. T. Springenberg, L. D. J. Fiederer et al., “Deep Learning with Convolutional Neural Networks for EEG Decoding and Visualization,” Human Brain Mapping, vol. 38, no. 11, pp. 5391–5420, 2017.

[9] K. K. Ang, Z. Y. Chin, H. Zhang and C. Guan, “Filter Bank Common Spatial Pattern (FBCSP) in Brain–Computer Interface,” Proceedings of the IEEE International Joint Conference on Neural Networks, pp. 2390–2397, 2008.

[10] A. Vaswani, N. Shazeer, N. Parmar et al., “Attention Is All You Need,” Advances in Neural Information Processing Systems 30, 2017.

[11] A. Gramfort, M. Luessi, E. Larson et al., “MEG and EEG Data Analysis with MNE-Python,” Frontiers in Neuroscience, vol. 7, 267, 2013.

[12] B. Aristimunha, I. Carrara, P. Guetschel et al., “Mother of All BCI Benchmarks (MOABB),” software framework for reproducible EEG decoding, 2023.

[13] G. Pfurtscheller and F. H. Lopes da Silva, “Event-Related EEG/MEG Synchronization and Desynchronization: Basic Principles,” Clinical Neurophysiology, vol. 110, no. 11, pp. 1842–1857, 1999.

[14] S. Holm, “A Simple Sequentially Rejective Multiple Test Procedure,” Scandinavian Journal of Statistics, vol. 6, no. 2, pp. 65–70, 1979.

[15] F. Wilcoxon, “Individual Comparisons by Ranking Methods,” Biometrics Bulletin, vol. 1, no. 6, pp. 80–83, 1945.